

(Ab)using Protocols

The Story of AntiDNS

William Dussault - 31/07/2025

About Me

- Developer @ Coveo since 2019
(opinions my own)
- I work on high-performance systems
 - Search engines & filesystems
 - C++, Rust & Go

What we'll talk about

- A DIY distributed filesystem
- A self-hosted video streaming service
- DNS

Menmos

“Where do I put my stuff?”

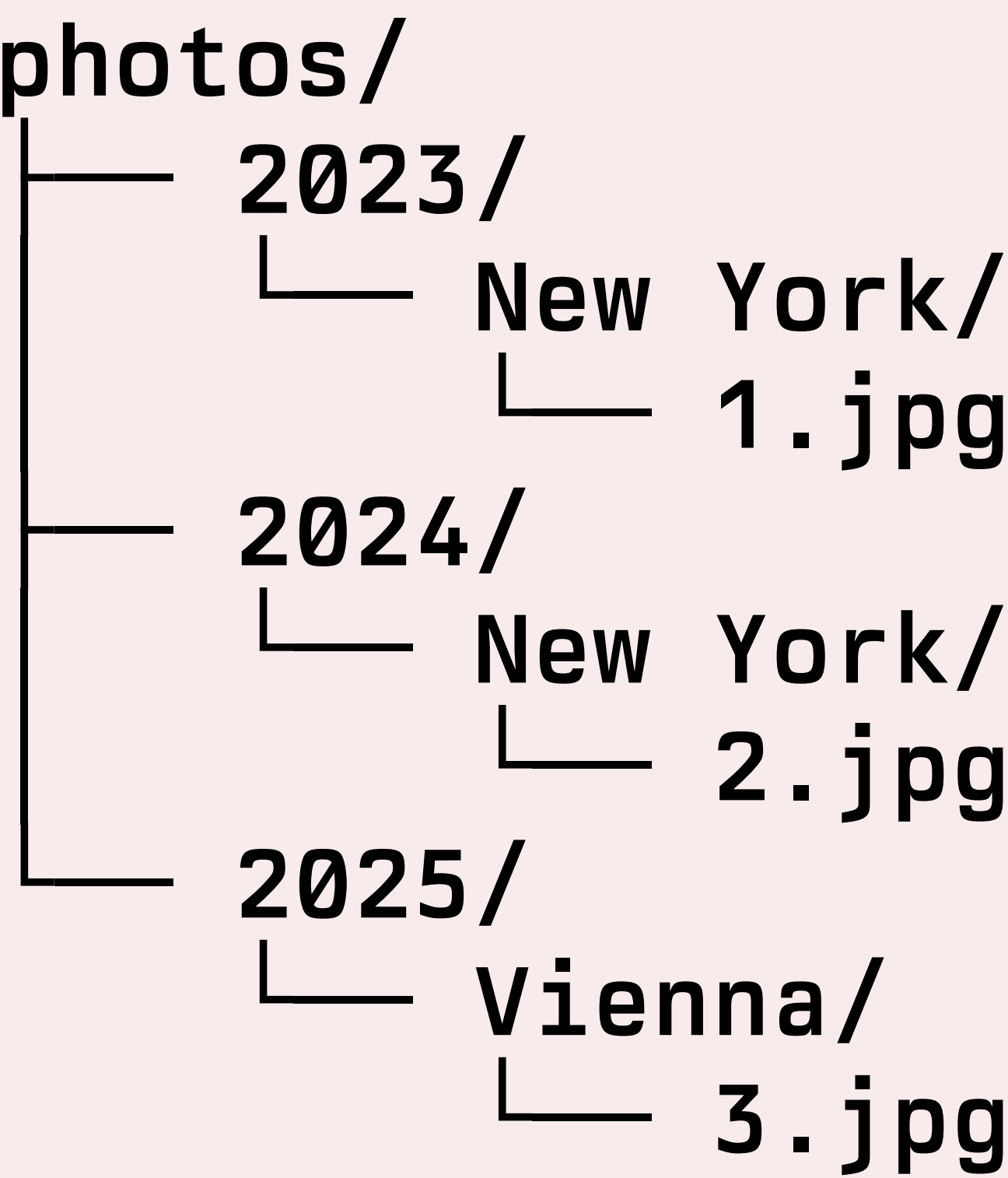
- Me

Hierarchical Filesystems Are Frustrating

```
graph LR; photos[photos/] --- 2023[2023/]; photos --- 2024[2024/]; photos --- 2025[2025/]; 2023 --- 1jpg[1.jpg]; 2024 --- 2jpg[2.jpg]; 2025 --- 3jpg[3.jpg];
```

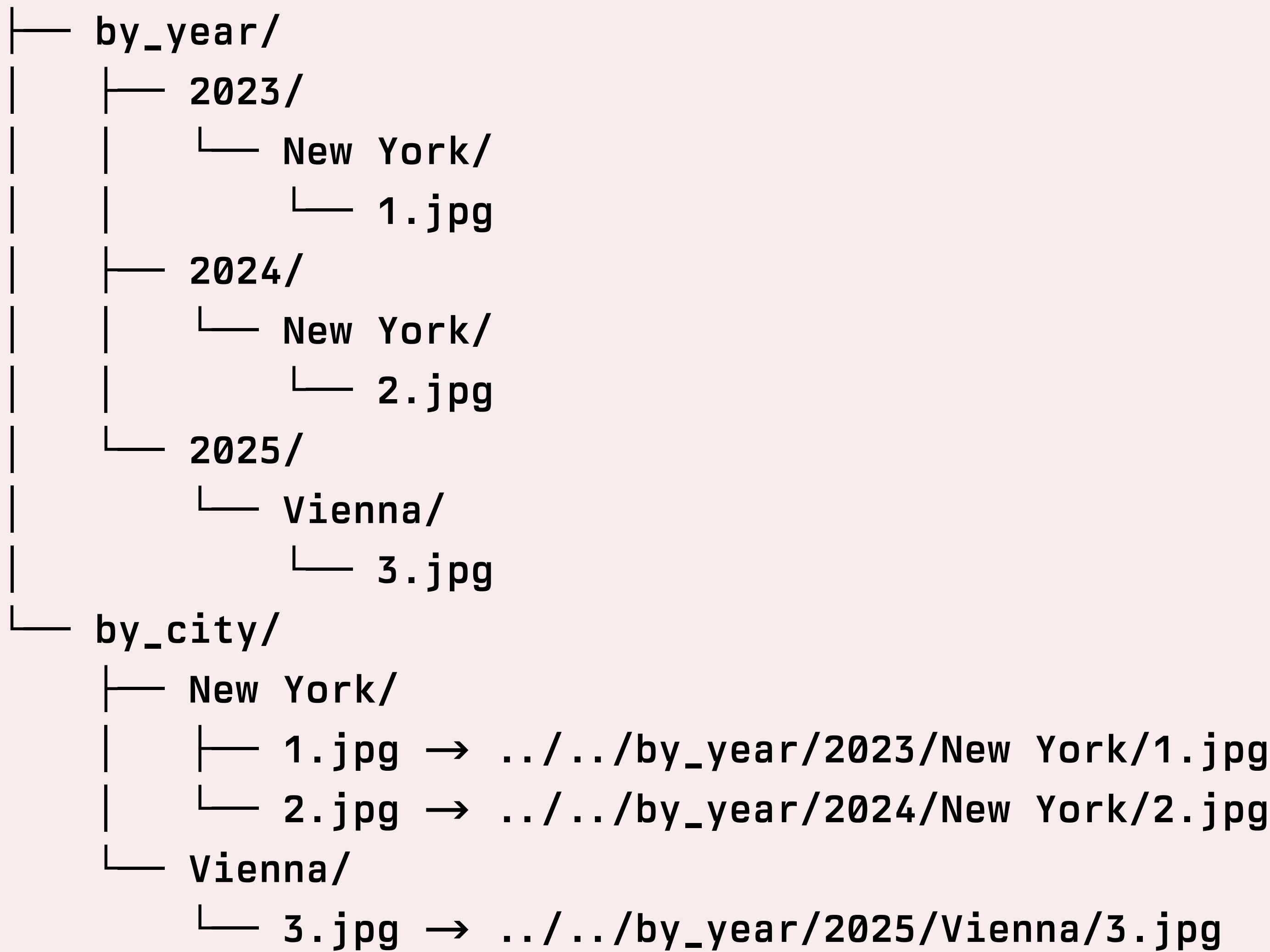
photos/
├── 2023/
│ └── 1.jpg
├── 2024/
│ └── 2.jpg
└── 2025/
 └── 3.jpg

Hierarchical Filesystems Are Frustrating



Hierarchical Filesystems Are Frustrating

photos/



A close-up of the character Yoda from Star Wars. He has a green, wrinkled face with large, expressive eyes. He is wearing his characteristic brown robes. The background is a dark, starry space. The text "I'll do it myself" is overlaid in a bold, black, sans-serif font across the middle of his face.

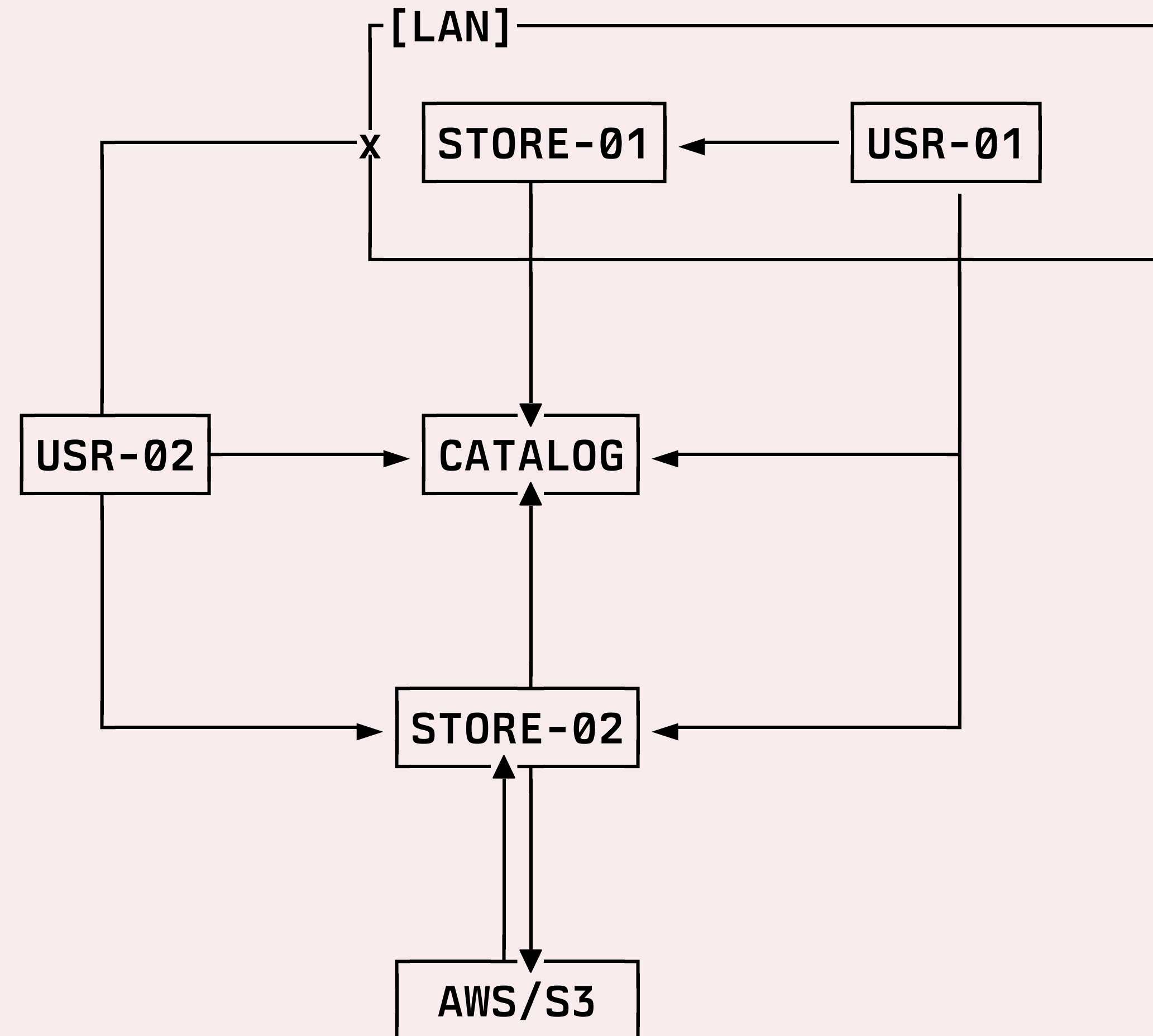
I'll do it myself

Requirements

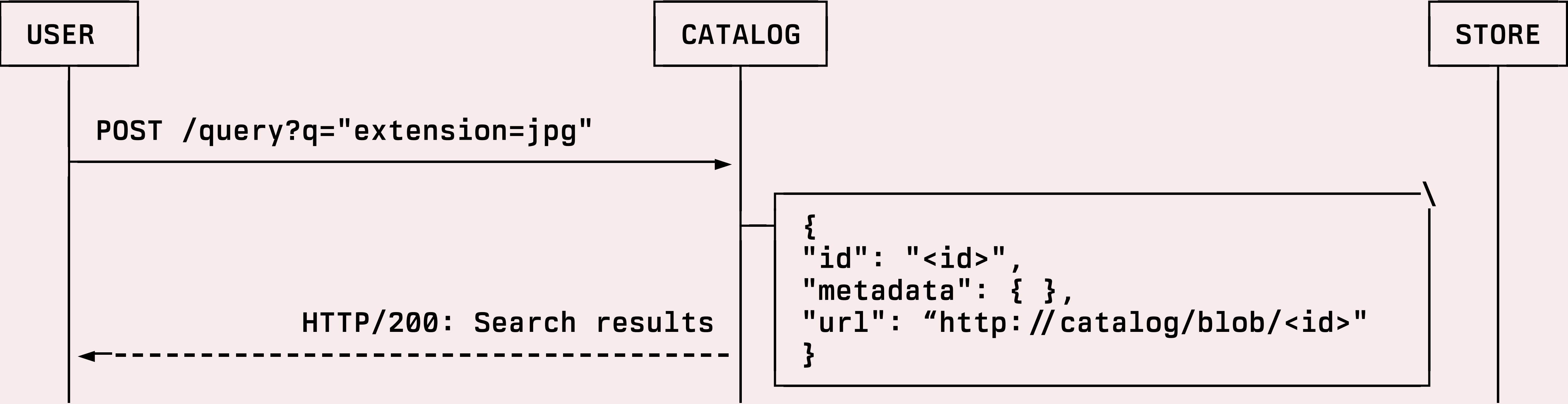
- Distributed storage
- One namespace
- Good search
- Minimize network trips

Design

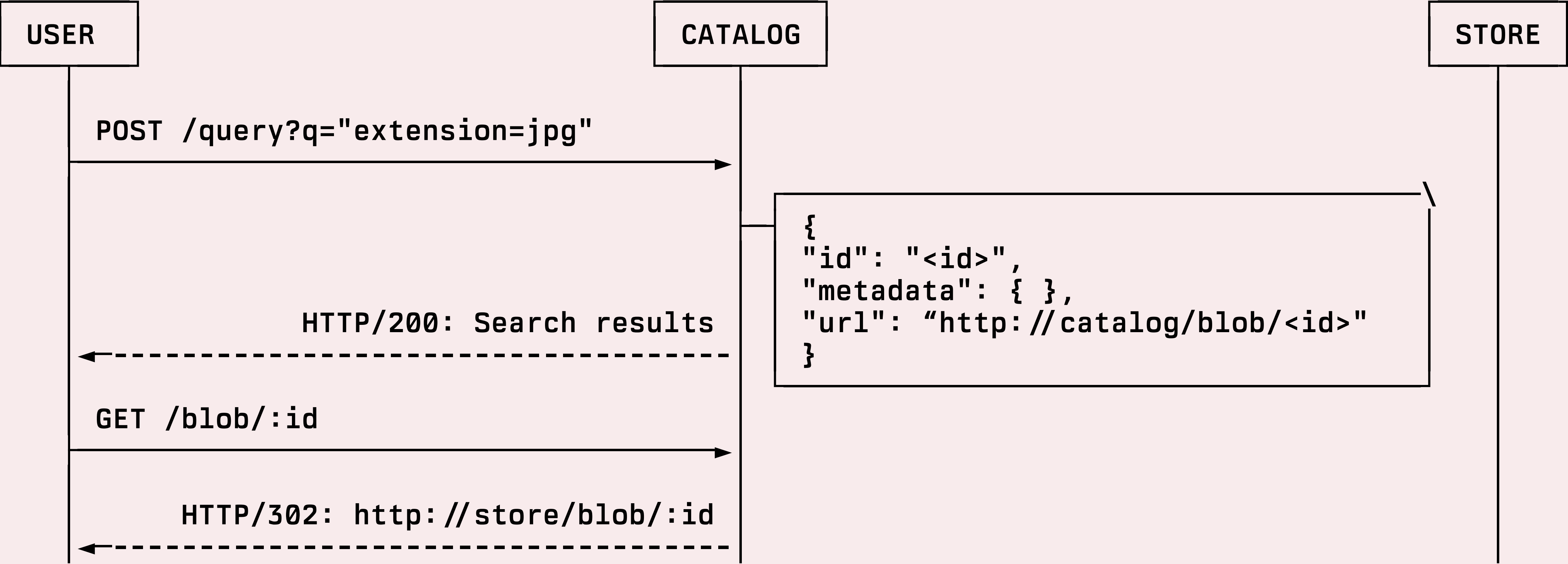
- Catalog cheap to host
- Doesn't change network access patterns



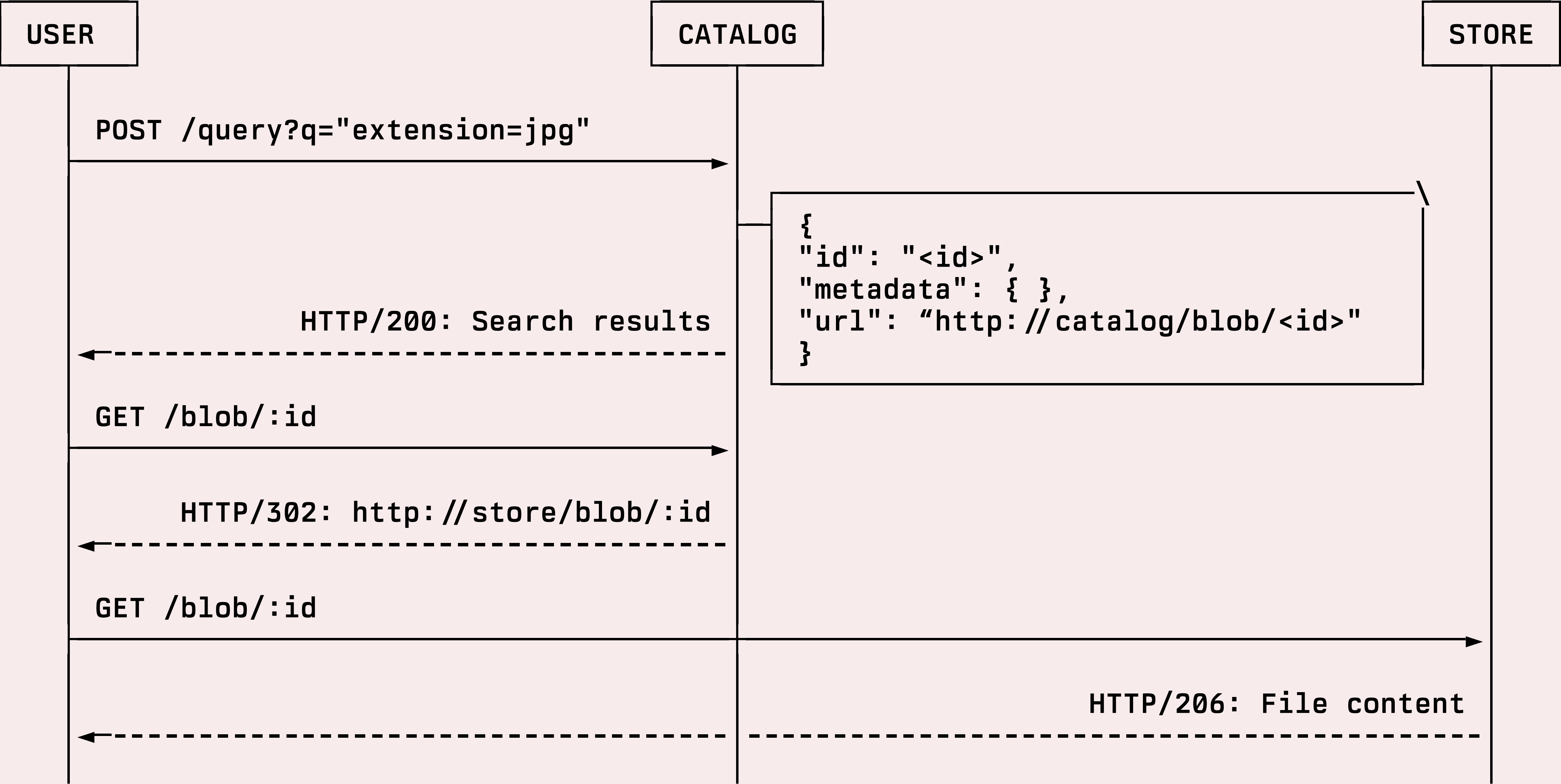
Query Process



Query Process



Query Process



The Problem

✖ ▶ Mixed Content: The page at '<https://dir.fs.menmos.xyz/>' was loaded over HTTPS, but requested an insecure image '<http://192.168.1.102/blob/asdf>'.

plex Media Server

- Home
- Movies
- TV Shows
- Music
- Shorts
- More >

Home

Home

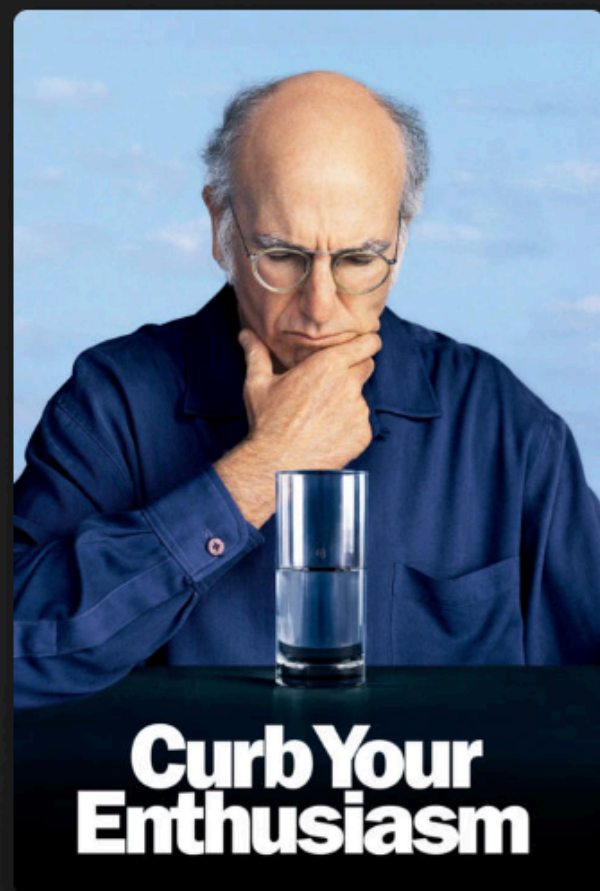
Trending

Activity

Find Friends

My Profile

Continue Watching



Curb Your Enthusiasm

The Ugly Section

S10 · E7



The Bear

Groundhogs

S4 · E1



The Rehearsal

Gotta Have Fun

S2 · E1



Breaking Bad

Negro y Azul

S2 · E7



It's Always Sunny in Phil...

The Gang F***s Up Abb...

S17 · E1

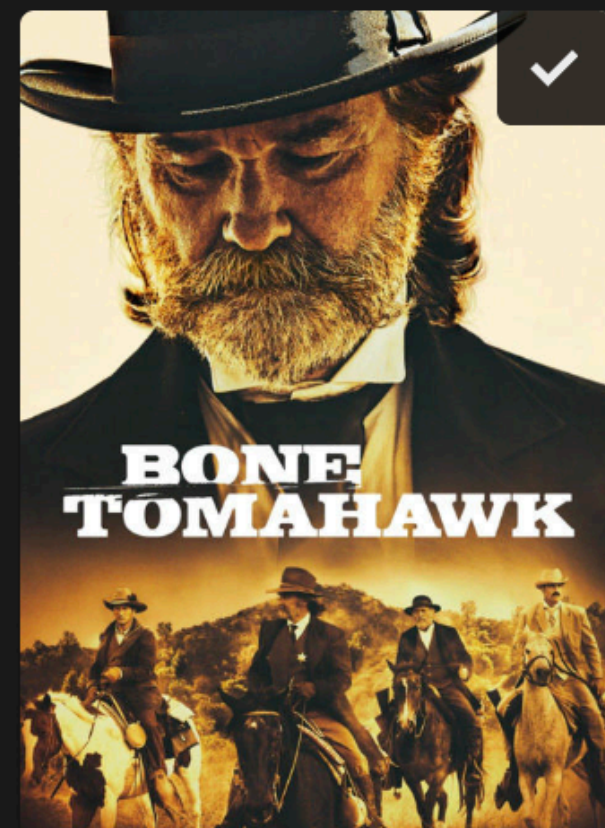


Formula 1: Drive to Surviv...

Frenemies

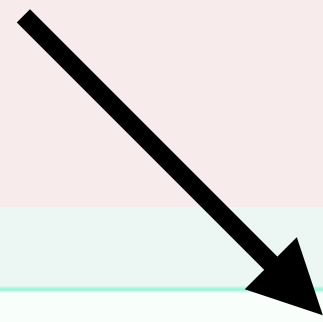
S7 · E2

Recently Added in Movies



But How?!

But How?!



▼ General	
Request URL	https://192-168-1-102.74b1a32303fb4c27a39f3560f1b2ccf7.plex.direct:32400/video/:/transcode/universal/session/ewp1mtzcfozzdvwaw17a41c8/0/7.m4s
Request Method	GET
Status Code	● 200 OK
Remote Address	192.168.1.102:32400
Referrer Policy	strict-origin-when-cross-origin

```
$ dig 192-168-1-102.74b1a32303fb4c27a39f3560f1b2ccf7.plex.direct
```



```
$ dig 192-168-1-102.74b1a32303fb4c27a39f3560f1b2ccf7.plex.direct

; <<>> DiG 9.10.6 <<>> 192-168-1-102.74b1a32303fb4c27a39f3560f1b2ccf7.plex.direct
;; global options: +cmd
;; Got answer:
;; —>HEADER<— opcode: QUERY, status: NOERROR, id: 18716
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;192-168-1-102.74b1a32303fb4c27a39f3560f1b2ccf7.plex.direct. IN A

;; ANSWER SECTION:
192-168-1-102.74b1a32303fb4c27a39f3560f1b2ccf7.plex.direct. 1546107 IN A 192.168.1.102

;; Query time: 5 msec
;; SERVER: 100.100.100.100#53(100.100.100.100)
;; WHEN: Mon Jul 28 18:28:57 EDT 2025
;; MSG SIZE rcvd: 103
```



```
$ dig 127-9-0-1.74b1a32303fb4c27a39f3560f1b2ccf7.plex.direct
```

```
$ dig 127-9-0-1.74b1a32303fb4c27a39f3560f1b2ccf7.plex.direct

; <<>> DiG 9.10.6 <<>> 127-9-0-1.74b1a32303fb4c27a39f3560f1b2ccf7.plex.direct
;; global options: +cmd
;; Got answer:
;; —>HEADER<— opcode: QUERY, status: NOERROR, id: 43546
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;127-9-0-1.74b1a32303fb4c27a39f3560f1b2ccf7.plex.direct.      IN A

;; ANSWER SECTION:
127-9-0-1.74b1a32303fb4c27a39f3560f1b2ccf7.plex.direct. 2592000 IN A 127.9.0.1

;; Query time: 199 msec
;; SERVER: 100.100.100.100#53(100.100.100.100)
;; WHEN: Mon Jul 28 18:32:51 EDT 2025
;; MSG SIZE rcvd: 99
```

```
$ dig +noall +answer soa plex.direct
```



```
$ dig +noall +answer soa plex.direct
```

```
plex.direct.          68      IN      SOA      ns-plexdirect.plex.tv. dns-admin.plex.tv.
```

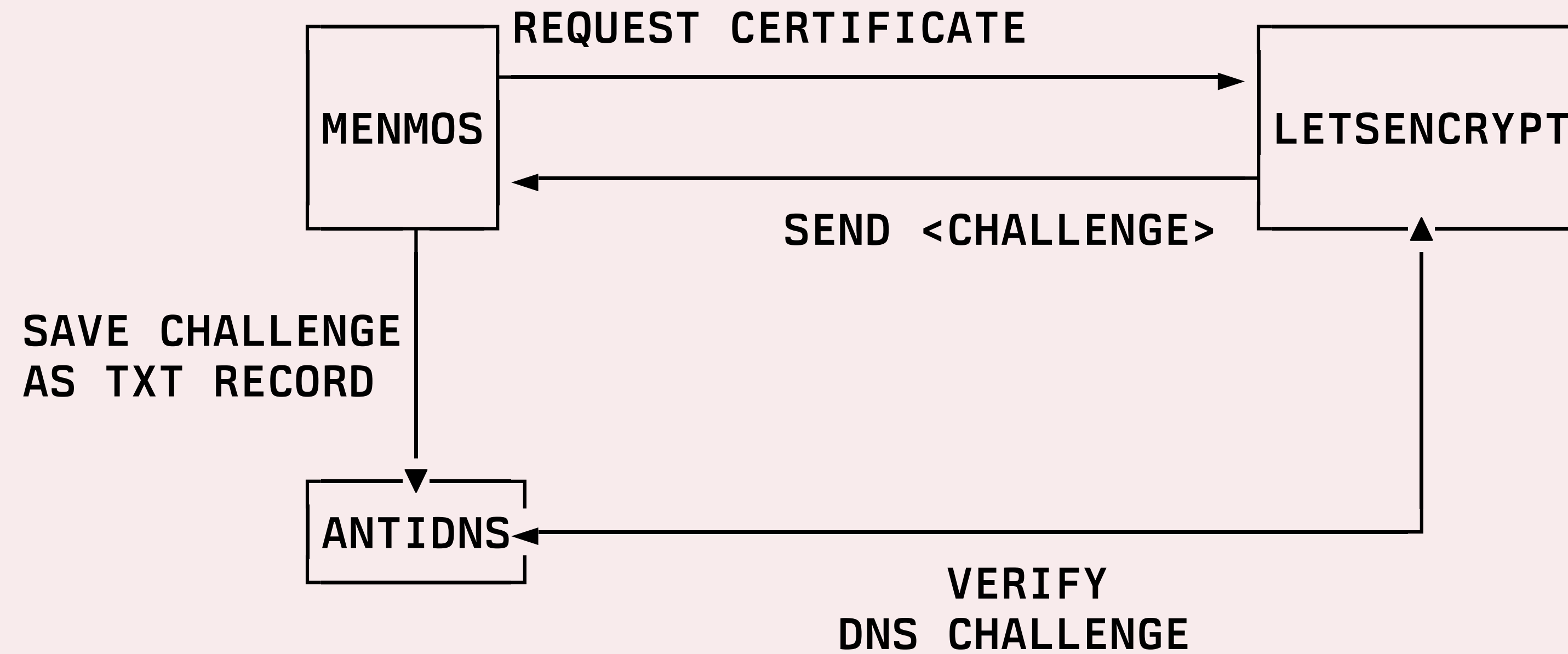



Back to Menmos

AntiDNS

```
pub async fn lookup(qname: &str, qtype: QueryType, cfg: &Config) → Result<Option<DnsPacket>> {  
    // Attempt to forward A queries that have a serialized IP in their domain to the IP itself.  
    if qtype == QueryType::A {  
        if let Ok(ip) = extract::ip_address_from_url(qname) {  
            return Ok(Some(in_house_resolution(qname, IpAddr::V4(ip))?));  
        } else if qname == cfg.host_name {  
            return Ok(Some(in_house_resolution(qname, cfg.public_ip)?));  
        }  
    }  
  
    // Necessary for LetsEncrypt DNS-01 challenge  
    // to work properly.  
    if qtype == QueryType::CAA {  
        // Here we do a very bad thing and don't really check or allow the possibility for CAA records.  
        // This means that any public CA has the authority to emit certificates for your domain.  
        // If you really _need_ CAA, you should probably be using something more serious than this  
        return Ok(Some(DnsPacket::new()));  
    }  
  
    // Otherwise we ditch the packet.  
    Ok(None)  
}
```

AntiDNS

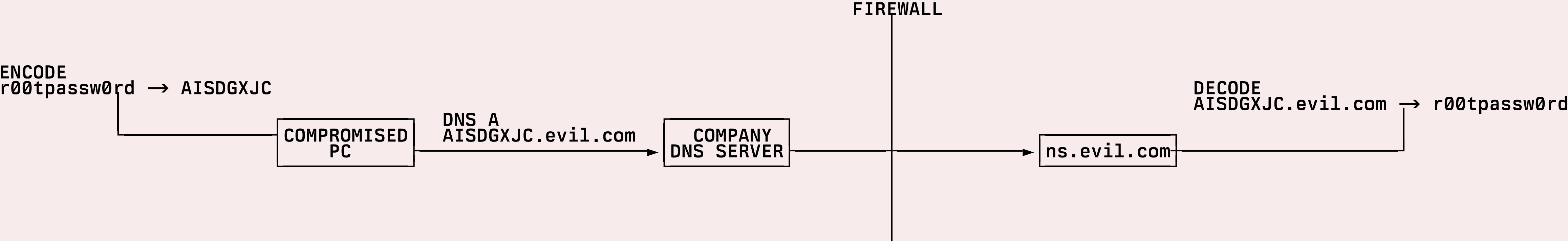


Demo

Why (Ab)use DNS?

- Allows idempotent & stateless dynamic DNS
- Works with LetsEncrypt without having to do backflips
- Scales to a large number of clients by using wildcard certificates
- Downside:
 - By itself, no authentication or validation of target IP possible

Bonus: DNS Exfiltration



**“Protocols are just tools,
How you use them is up to you.”**

**THANK
YOU**